

TinyML HAR no ESP32-S3.

Inferência local em microcontrolador usando dataset público e modelo compacto.

DISCIPLINA

IA Embarcada e
Modelos Compactos

INTEGRANTES

Renan Mocelin
Nathan Arrais
Lucas Porfirio

INSTITUIÇÃO

Senai

Reconhecer o que uma pessoa faz, a partir de poucos sensores.

HAR (*Human Activity Recognition*) classifica atividades cotidianas usando aceleração triaxial — caminhar, sentar, deitar — sem câmera e sem nuvem.

→ SAÚDE

Monitoramento de mobilidade, quedas e sedentarismo em pacientes.

→ WEARABLES

Pulseiras e relógios que detectam atividade física e contexto.

→ SEGURANÇA

Deteção de queda, postura e fadiga em ambientes industriais.

→ MONITORAMENTO REMOTO

Inferência local preserva privacidade e funciona sem internet.

Objetivo: fazer essa inferência dentro de um microcontrolador, sem servidor.

Os requisitos

REQUISITO	COMO FOI ATENDIDO	
Dataset público	UCI HAR — 7.352 amostras de treino, 2.947 de teste.	✓
Sensor inercial	MPU6050 via I2C, aceleração triaxial (compatível com total_acc).	✓
Treinamento offline	Python com scikit-learn, MLP 43-32-6 com ReLU.	✓
Compressão do modelo	Quantização de float32 para int8 — redução de 75% nos pesos.	✓
Deploy embarcado	ESP-IDF em C, ESP32-S3 simulado no Wokwi.	✓
Inferência local	Pipeline completa no chip, classe prevista no monitor serial.	✓

UCI HAR — seis atividades, 50 Hz, janelas de 128 amostras.

Conjunto público *Human Activity Recognition Using Smartphones*, coletado com smartphone na cintura e usado como base de treino e teste.

TREINO

7.352

amostras

TESTE

2.947

amostras

JANELA

128

leituras / janela

FREQUÊNCIA

50 Hz

amostragem

SEIS CLASSES

WALKING

WALKING_UPSTAIRS

WALKING_DOWNSTAIRS

SITTING

STANDING

LAYING

Subconjunto utilizado: `total_acc_x` · `total_acc_y` · `total_acc_z`

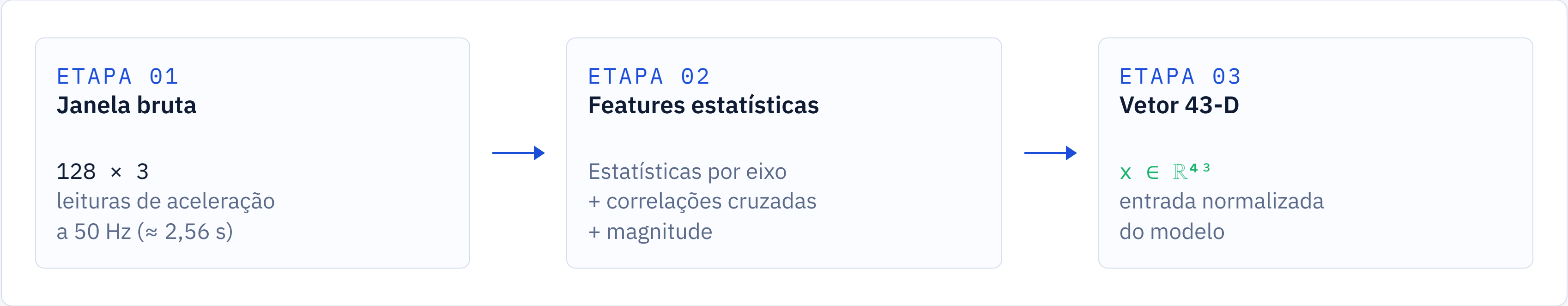
Do smartphone na cintura ao MPU6050 no Wokwi.

O dataset original foi coletado com smartphone, mas o sinal usado é aceleração triaxial — exatamente o que o MPU6050 fornece. A pipeline de features é a mesma.



Observação: a simulação no Wokwi demonstra a pipeline embarcada; o modelo foi treinado apenas com o dataset público.

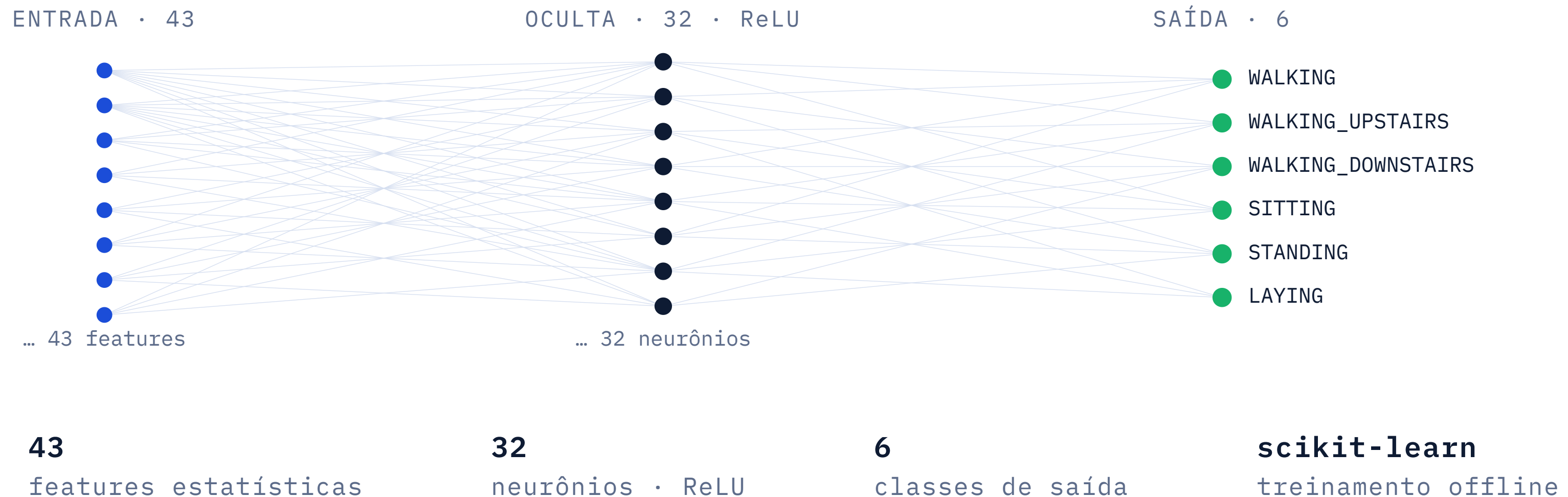
De 128 leituras brutas a um vetor de 43 features.



média	desvio padrão	mínimo	máximo
RMS	x médio	amplitude	diferenças
cruzamento por zero	correlação x·y	correlação x·z / y·z	magnitude da aceleração

MLP 43-32-6 — pequena por desenho.

Uma única camada oculta com ativação ReLU. Treinada em Python com `scikit-learn`, exportada para um header C.



Quantização para int8: **–75% de memória**, acurácia preservada.

MODELO ORIGINAL · FLOAT32

85,78%

Tipo dos pesos **float32**

Tamanho relativo **4× int8**



100% do baseline

MODELO EMBARCADO · INT8

85,71%

Tipo dos pesos **int8**

Tamanho relativo **1× baseline / 4**



25% do baseline · **–75%**

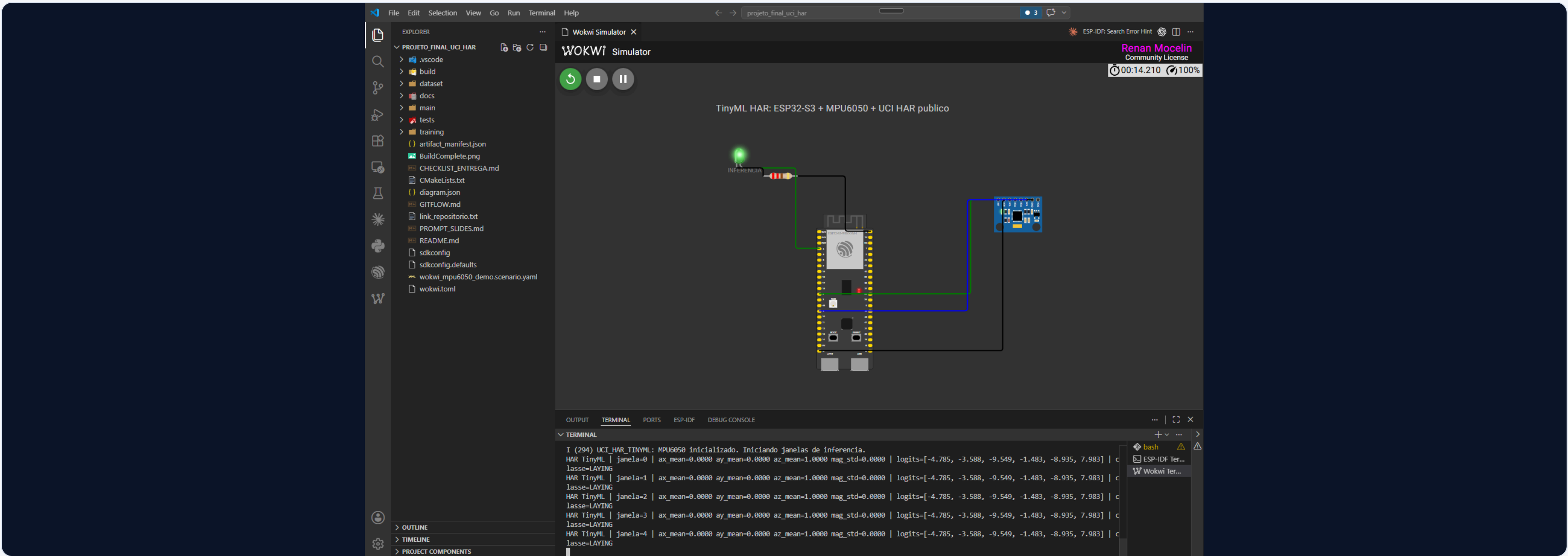
Δ acurácia: **–0,07 ponto percentual** – perda mínima diante de 4× menos memória de pesos.

Pipeline embarcada em C: **sensor** → **features** → **MLP int8** → **classe**

- **Aquisição.** Leitura I2C do MPU6050 e montagem da janela de 128 amostras × 3 eixos.
- **Features.** Extração das 43 estatísticas no próprio chip, sem buffers extras.
- **Normalização.** Aplicada com parâmetros exportados do treinamento.
- **Inferência.** MLP 43-32-6 executada com pesos quantizados em int8.
- **Saída.** Classe prevista impressa no monitor serial; LED indica execução.

```
01 main/main.c // loop principal
02 main/har_feature_extraction.h // 43 features
03 main/har_inference.h // MLP int8
04 main/har_model_int8.h // pesos exportados
05 diagram.json // circuito Wokwi
06 wokwi.toml // config Wokwi
//
07 training/train_uci_har_accel_model.py treino
sklearn
```

ESP32-S3 + MPU6050 + LED indicador, no Wokwi.



MPU6050 → ESP32-S3
SDA • GPIO 8

MPU6050 → ESP32-S3
SCL • GPIO 9

LED INDICADOR
GPIO 4 • 220 Ω

SAÍDA
Serial • HAR TinyML

Build concluído, modelo quantizado e teste C nativo passando.

ACURÁCIA EM TESTE · INT8

85,71%

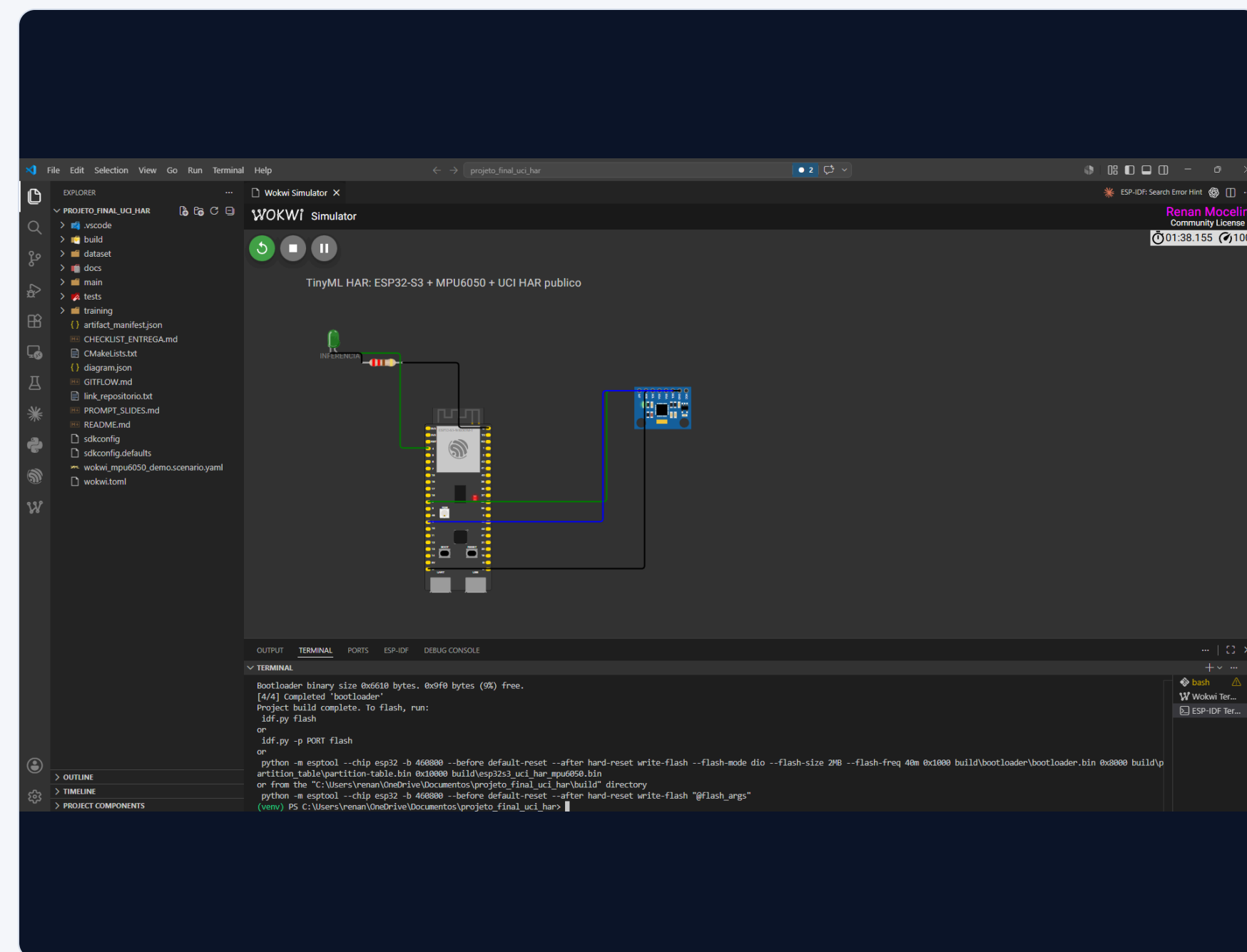
vs. 85,78% em float32 ($\Delta -0,07$ pp)

DATASET UTILIZADO

UCI HAR · público

7.352 treino · 2.947 teste

- **Build ESP-IDF.** Project build complete.
- **Imagem.** ESP32-S3 gerada com checksum válido.
- **Execução.** Monitor serial imprime linhas HAR TinyML com a classe.
- **Teste C nativo.** Casos reais das 6 classes passam.



O que ficou de fora — e como o projeto pode crescer.

LIMITAÇÕES

- **Movimento simulado.** O Wokwi não reproduz movimento corporal real com fidelidade.
- **Gap de sensor.** Dataset foi coletado com smartphone na cintura, não com MPU6050.
- **Apenas aceleração.** O giroscópio do MPU6050 não foi utilizado nesta versão.

PRÓXIMOS PASSOS

- **Dados próprios.** Coletar amostras reais usando o MPU6050.
- **Mais sinais.** Incluir o giroscópio para features rotacionais.
- **Retreinar.** Atualizar o modelo com o dataset próprio.
- **Comparar runtimes.** Avaliar TFLite Micro frente à MLP int8 manual.